MuseRelay AWS Engagement Bridge — one AI agent, WhatsApp and web chat, one ticket

AWS CDS Agentic AI Partner Hackathon 2026 · Screen Art SL · Architecture as deployed (eu-west-1 for the Bridge, EU inference for Amazon Bedrock)

Customer on WhatsApp
real WhatsApp number

WhatsApp

1

AWS Cloud

AWS Engagement Bridge — NEW module built for the hackathon (eu-west-1)

AWS SAM / CloudFormation · open source: github.com/screenartsl/aws-engagement-bridge

AWS End User Messaging Social
WhatsApp Business phone number on AWS
(AWS CDS)

Amazon SNS → Amazon SQS → Inbound Lambda

3

Sender Lambda ← Outbound SQS ← Enqueue Lambda + HTTP API

AWS SDK: SendWhatsAppMessage, PostWhatsAppMessageMedia

Supporting AWS services

- Amazon DynamoDB — idempotency, duplicate protection, durable state
- AWS Secrets Manager — HMAC signing keys per binding and direction
- AWS KMS — encryption of topics, queues and media · Amazon S3 — WhatsApp media
- SQS dead-letter queues · Amazon CloudWatch Logs (restricted content)
- Separate IAM execution role per Lambda · no AWS access keys in the runtime
- No fallback to the direct Meta API when the AWS transport fails

Amazon Bedrock (EU cross-region inference)

- Claude Haiku 4.5 — reasoning and tool use for the MuseRelay agent
- Amazon Nova Micro — guard classifiers

Called by the MuseRelay Core (Converse API with tools). The model proposes; the server validates every tool call and business rule.

signed HTTPS

signed outbound API

code via WhatsApp

Converse + tools

MuseRelay Core — pre-existing SaaS (EU servers), extended

Signed connector + native unified inbox
verifies HMAC signature, expiry and replay · contacts, conversations, messages and WhatsApp delivered / read status

One AI agent for every channel ("Screen Art Demo") 2
same agent, prompt and tools answer WhatsApp and the web chatbot; intent-based and multilingual; tickets belong to the customer, not to the channel

Support ticket tools (server-enforced)
create (only after the customer confirms) · list my tickets · add information · update status · audit log of every change
public tracking link with an unguessable 48-character token

Cross-channel verification (one-time code) 5
customer gives the reference, or the email / phone used to open it
→ 6-digit code sent ONLY to the contact already on the ticket
→ checked by the server before the model (the AI never sees it)
→ that conversation may consult and update the same ticket for 12 h
Neutral answers · 10 min, single use, attempt limits · central send limits per requester, recipient, organization, reseller and platform

same agent

Human supervision 6
operators work from the inbox and the ticket page; replying from the ticket pauses the AI in that conversation; internal notes are kept separate and never sent to the customer

Web chatbot endpoint
widget embedded on the demo site; HTTPS to the Core (does not use the AWS transport)

Customer email inbox
platform mail service (not Amazon SES)

one-time code by email (web → WhatsApp direction)

Customer on web chat
chatbot embedded on demo.screenart.es

4

HTTPS chat (reference, or email / phone, then the code)

Flow 1 WhatsApp message in through AWS · 2 Agent reasons with Claude Haiku 4.5 and uses ticket tools (ticket created after confirmation) · 3 Reply out through the Bridge (AWS SDK)
4 Same customer continues on the web chat · 5 One-time code to the contact on the ticket (WhatsApp via the Bridge, or email), checked by the server; same ticket updated · 6 Human operator takes over

→ verified live (27–28 Oct 2026) ⇢ implemented and tested, live check pending

Not shown as operational: SMS (dry run) · Amazon SES for ticket emails (SES proven separately; codes use the platform mail service)